SEC-RAG: A Retrieval-Augmented Generation System for Financial Question Answering over SEC 10-Q Filings

---

CS 5170 - Assignment 2 | Kevin Xu, Sheldin Lau, Eric Trinh

Abstract

SEC-RAG is an end-to-end retrieval-augmented generation (RAG) system designed for high-precision factual question answering over SEC 10-Q filings. Focusing on the top 30 NASDAQ technology companies by market capitalization, the system processes fiscal year 2025 disclosures from the SEC EDGAR database. To overcome the limitations of standard text-splitting in financial contexts, we implement a structure-aware HTML parser that transforms complex financial tables into queryable natural language sentences via fiscal calendar normalization. The architecture utilizes the BAAI/bge-base-en-v1.5 model for dense vector retrieval within a FAISS index, coupled with Qwen2.5-7B-Instruct as the generative reader. While our evaluation shows an F1 score of 30% and a recall of 42%, qualitative analysis reveals that the system successfully identifies relevant context but struggles with precise token-level extraction. We identify key bottlenecks in LLM reasoning, including temporal misalignment across fiscal quarters and "context overflow" hallucinations, and propose strategies for improving entity-aware filtering and table representation.

Introduction

Large language models (LLMs) have demonstrated strong performance on general question answering tasks (Touvron et al., 2023), but their static parametric knowledge places a hard ceiling on domains requiring precise, up-to-date, or highly specialized information. Financial reporting is a canonical example of such a domain: SEC 10-Q quarterly filings contain exact revenue figures, earnings per share, segment breakdowns, and management commentary that change every quarter and are unlikely to be reliably encoded in a model's weights.

Retrieval-Augmented Generation (Lewis et al., 2021) addresses this limitation by augmenting a language model with a retrieval step that fetches relevant passages from an external knowledge base at inference time. The model is then conditioned on both the query and the retrieved context to produce its answer. This approach is particularly well-suited to financial QA, where the answer is often a precise figure that exists verbatim in a document rather than as a synthesized judgment. Building a RAG for SEC filings is not without its challenges. The documents provided by EDGAR are rich in metadata, but inconsistent formatting across different companies and the use of legacy

systems create additional complexity in parsing the documents. LLMs also have trouble with interpreting information in tables, so additional processing is necessary when parsing tables.

We build SEC-RAG to answer factual questions about the financials of major technology companies as reported in their most recent 10-Q filings. The system must handle the unique challenges of financial documents: dense tables with complex multi-period headers, fiscal calendar heterogeneity across companies, and the need for exact numerical extraction rather than approximate semantic matching.

Domain and Data

A. *Domain Choice*

Our chosen domain is quarterly financial reporting for the top 30 NASDAQ technology companies by market capitalization. The target companies include Apple (AAPL), Microsoft (MSFT), NVIDIA (NVDA), Amazon (AMZN), Meta (META), Alphabet (GOOGL), Tesla (TSLA), Broadcom (AVGO), and 22 others spanning semiconductors, cloud infrastructure, enterprise software, and consumer internet. All 10-Q filings are publicly available through the SEC EDGAR system under mandatory disclosure requirements. RAG is effective for this domain for three reasons. First, exact financial figures (e.g., "Apple's iPhone revenue for Q3 FY2025") update every quarter and the time-sensitive nature of the data makes a retrieval system appropriate. Second, the sheer specificity of the queries (particular line items for particular companies in particular fiscal periods) makes retrieval a prerequisite for accurate grounding. Third, the formatting of financial tables in HTML filings requires specialized parsing that is orthogonal to LLM pretraining.

B. *Knowledge Resource*

We collected 10-Q filings filed with the SEC EDGAR system between January 1, 2025 and December 31, 2025 for all 30 target tickers. The documents were retrieved from the SEC EDGAR website (data.sec.gov/submissions/). The documents are in html format. The documents often come with additional reference documents and attachments, but we filtered for only the primary 10-Q document for our system. The final knowledge resource contains approximately 90 10-Q filings in HTML format (3 per company depending on filing cadence within 2025), constituting the primary 10-Q documents for each company. These documents collectively contain income statements, balance sheets, cash flow statements, stockholders' equity statements, and Management's Discussion and Analysis (MD&A) sections.

C. *Test Data Annotation*

We used a question set of 160 question and answer pairings based on information that can be found directly in the 10-Q documents. The questions and answers are generated using Gemini and include questions about

financial metrics in specific quarter filings. We also included questions that are found outside of tables and questions that have non-numeric answers.

System Description

A. *Architecture Overview*

SEC-RAG follows the standard RAG pipeline: (1) offline document indexing and (2) online query answering. The pipeline is:

SEC EDGAR download → HTML parsing & chunking → FAISS vector index → dense retrieval → Qwen-2.5 generation

All components are implemented using LangChain (Chase, 2022), HuggingFace Transformers (Wolf et al., 2020), and the FAISS library (Johnson et al., 2019).

B. *Document Download*

Filing acquisition is handled by download_10q.py , which queries the SEC EDGAR submissions API to enumerate 10-Q filings for each of the 30 target tickers within a user-specified date range. For each filing, the script parses the SEC index page (HTML tableFile) to locate the primary 10-Q document URL, stripping the iXBRL inline viewer wrapper when present. Files are saved locally under a sec-edgar-10q-filings/{TICKER}/ directory structure.

C. *Document Parser*

Standard text splitters are insufficient for financial filings because they destroy the row-column structure of financial tables. We implement a custom structure-aware parser (run.py) built on BeautifulSoup4 (Richardson, 2007) that treats HTML table elements and text elements differently.

Table parsing: For each table element, the parser extracts rows, detects period headers matching patterns like "Three Months Ended" or "Six Months Ended" (possibly split across consecutive rows), and rewrites them as normalized fiscal quarter labels (e.g., "Q3 FY2025 vs. Q3 FY2024") using a per-ticker fiscal year end calendar. Tables with detected period headers are then converted row-by-row into natural language sentences of the form:

[AAPL 10-Q 2025-08-01] Net sales: 94,930 (Q3 FY2025) vs 85,777 (Q3 FY2024)

This "table-to-NL" conversion improves semantic retrieval by exposing each metric as a standalone, queryable sentence rather than requiring the model to parse raw markdown or HTML. Tables without period headers are stored as markdown blocks and split only when they exceed 10,000 characters.

Text parsing: Narrative text from <p> , , and <li> elements (excluding those inside tables) is extracted, cleaned (normalizing spacing around $ and %), and split with a

RecursiveCharacterTextSplitter using chunk_size=1,500, chunk_overlap=200.
Metadata tagging. Each document chunk is tagged with: ticker , filing_date , form (always "10-Q"), content_type (table_nl , table , or text), section (matched against 30+ regex patterns covering income statement, balance sheet, MD&A, notes, etc.), note_number , and period (fiscal quarter of the chunk).

Noise filtering: Chunks tagged as signature , table_of_contents , or cover_page are discarded.

D. *Embedding and Vector Store*

Document chunks are embedded using BAAI/bge-base-en-v1.5 (768-dimensional, normalized), a strong general-purpose bi-encoder known for dense retrieval tasks (Xiao et al., 2023). Embeddings are computed on GPU with batch_size=32. The resulting vectors are stored in a FAISS flat index using cosine similarity (DistanceStrategy.COSINE). The index is persisted to disk for reuse.

E. *Retriever*

At query time, the query is embedded with the same bge-base-en-v1.5 model and the top k=5 nearest chunks are retrieved by cosine similarity. No metadata pre-filtering is applied in the final submitted system (ticker-based filtering was implemented but disabled after it was found to reduce recall on questions that matched cross-company context).

F. *Reader / Generator*

Retrieved chunks are concatenated and passed as context to two compact instruction-tuned causal language models optimized for reasoning and instruction following: microsoft/Phi-3-mini-4k-instruct and Qwen2.5-7B-Instruct, both running in float16 on GPU. The same prompt is used for both models, instructing each to answer using only information explicitly stated in the context, report exact figures and units as they appear in the filing, and decline with "This information is not available in the provided 10-Q" when the answer is absent. Generation uses temperature=0.1 and max_new_tokens=512 for both models. However, post-processing differs between the two: because Phi-3-mini outputs may contain model-specific special tokens (<|thinking|>, <|...|>), a custom output parser is applied to strip these tokens from its responses, whereas Qwen2.5-7B-Instruct outputs require no such filtering. For our baseline, we performed queries using both Qwen2.5-7B-Instruct and Phi-3-mini-4k-instruct without the use of document retrieval.

Results

A. *Evaluation Metrics*

We evaluate using the standard SQuAD metrics (Rajpurkar et al., 2016): Exact Match (EM), F1,

and Recall, all computed at the token level after lowercasing and removing articles and punctuation. For questions with multiple valid reference answers (semicolon-separated), the maximum score across all valid answers is taken.

B. *Quantitative Results*

Output using Qwen:

```
--- Evaluation Results ---
  exact_match          0.0%       predictions that matched a reference answer word-for-word
  f1                   30.52%     average token overlap between predictions and reference answers (main metric)
  recall               42.39%     reference answer tokens successfully captured in predictions
  num_questions        160        total questions evaluated
--------------------------
```

The system did not score well on exact match, recording **0.0%** — indicating that the model never reproduced a reference answer word-for-word, which is expected given that instruction-tuned models tend to generate verbose or paraphrased responses rather than terse, precise extractions, even when instructed otherwise. The system achieved an **F1 score of 30.52%** and a **recall of 42.39%**, suggesting it is able to retrieve relevant chunks of information but often fails to locate the precise information requested. The stronger recall in particular implies that Qwen2.5-7B is fairly thorough in covering reference answer tokens when the right context is retrieved, but precision remains limited by the model's tendency to paraphrase rather than extract exact spans.

```
--- Evaluation Results ---
  exact_match          0.0%       predictions that matched a reference answer word-for-word
  f1                   28.0%      average token overlap between predictions and reference answers (main metric)
  recall               32.95%     reference answer tokens successfully captured in predictions
  num_questions        160        total questions evaluated
--------------------------
```

The model achieved an **F1 score of 28.0%**, which serves as the primary metric reflecting average token overlap between the model's predictions and the reference answers. The **recall of 32.95%** suggests the model was able to surface roughly a third of the relevant tokens from the reference answers, meaning it captured some of the right information but often produced incomplete or partially misaligned responses. The **exact match score of 0.0%** indicates that the model never reproduced a reference answer word-for-word, which is expected given that Phi-3-mini tends to generate verbose or paraphrased responses rather than terse, precise extractions — even when instructed otherwise.

Results with Qwen/Qwen2.5-7B-Instruct no RAG

```
--- Evaluation Results ---
  exact_match          0.0%       predictions that matched a reference answer word-for-word
  f1                   15.95%     average token overlap between predictions and reference answers (main metric)
  recall               54.92%     reference answer tokens successfully captured in predictions
  num_questions        160        total questions evaluated
--------------------------
```

Results with microsoft/Phi-3-mini-4k-instruct no RAG

```
--- Evaluation Results ---
  exact_match          0.0%       predictions that matched a reference answer word-for-word
  f1                   15.75%     average token overlap between predictions and reference answers (main metric)
  recall               55.26%     reference answer tokens successfully captured in predictions
  num_questions        160        total questions evaluated
--------------------------
```

Both models performed very similarly without RAG, with Qwen2.5-7B-Instruct achieving an **F1 of 15.95%** and recall of **54.92%**, while Phi-3-mini recorded an **F1 of 15.75%** and recall of **55.26%** — a negligible difference across both metrics. The exact match remained **0.0%** for both, which is unsurprising given the nature of 10-Q filings, which are dense with highly specific numerical figures, dates, and financial terminology that are virtually impossible to reproduce accurately from parametric memory alone. The high recall relative to F1 in both cases suggests the models are generating responses that contain some overlapping tokens with the reference answers, but without access to the actual document, they cannot reliably pinpoint the precise figures and values requested. This highlights the core motivation for the RAG pipeline — financial filings require grounding in the source document, and even capable instruction-tuned models fall short when forced to rely solely on what they may have seen during pretraining.

Analysis

A. *Qualitative Analysis*

In our provided annotated_output.csv file, we have noted the possible failures and reasoning for failed incorrect responses to the test set. SEC-RAG succeeded in finding relevant chunks based on similarity score, but it often fails to accurately extract the specific numerical figure requested.

The most frequent mistake is extracting the right metric but for the wrong period of time. This is likely a combination of two separate issues. The first is that for the simplicity of our test set generation, we used Q1-Q3 in our prompts, but each company can define different time periods for their fiscal year which could differ drastically from each other and from the calendar year. To handle this, we need extensive metadata tagging on each document so that they can accurately reflect each company's fiscal years and an LLM that can handle that nuance. The second issue is that some of the tables have complex headers that are difficult for LLMs to interpret, even after converting the tables to markdown format. The table below shows one such table found in the financial statement for Apple. Not only is there an additional subheader, the headers and subheaders have different dimensions. To address this, the parser would need to further reformat the tables by

either rewriting the headers into a simpler format or even converting them to json format.

	Three Months Ended		Six Months Ended	
	March 29, 2025	March 30, 2024	March 29, 2025	March 30, 2024
Net sales:				
Products	$ 68,714	$ 66,886	$ 166,674	$ 163,344
Services	26,645	23,867	52,985	46,984
Total net sales	95,359	90,753	219,659	210,328

Another common error is that queries about revenue would often return $42.3 billion, which is Meta's Q1 revenue for 2025. This is likely due to a problem with how relevancy is scored when retrieving chunks. The word revenue being repeated a high number of times in Meta's filing lead to that particular chunk scoring exceptionally high during retrieval. This can probably be fixed with modifications to relevancy score to add higher priority to company name or ticker when calculating k-nearest neighbor and by increasing the number k so that the relevant chunk is more likely to be retrieved.

Sometimes semantically similar terms can cause the incorrect chunk to be retrieved due to the use of dense retrieval. When asked about the revenue for Analog Devices, SEC-RAG returned the total revenue for Texas Instruments' analog segment revenue. If the use case of the RAG can be reduced to retrieval of numerical figures, it would be advantageous to swap to sparse retrieval, where more importance is given to exact keyword matches.

B. *Closed-Book vs. RAG Analysis*

We qualitatively assessed the value of retrieval by running two questions against Phi-3 without context (closed-book):

"What was Apple's diluted EPS for Q3 FY2025?" Closed-book: refused or produced a stale/incorrect figure. RAG: "This information is not available in the provided 10-Q" (false negative, but at least not hallucinated).

"By what percentage did Azure grow in Q3 FY2025?" Closed-book: no knowledge. RAG: "Server products and cloud services revenue increased 22% driven by Azure and other cloud services revenue growth of 33%" (correct 33% figure present but embedded in verbose output).

This confirms that retrieval is necessary: closed-book Phi-3 has no knowledge of 2025 financial figures. The system's failures are primarily generation-side (verbosity, hallucination under overflow) rather than retrieval-side misses.

Conclusion

We have demonstrated the implementation of SEC-RAG, a specialized RAG pipeline tailored to the structural and temporal nuances of quarterly financial reporting. By integrating a novel table-to-NL conversion strategy and fiscal calendar normalization, our system attempts to bridge the gap between semi-structured HTML filings and LLM comprehension. Our findings underscore a critical reality in financial AI: while dense retrieval models (like BGE-base) are effective at surfacing relevant documents, the "last mile" of extraction remains a significant challenge. Performance was primarily hindered by generation-side issues—specifically, the model's difficulty in distinguishing between similar metrics across different time periods and a tendency toward over-conservative "not available" responses.

To evolve SEC-RAG into a production-grade tool, future iterations should prioritize:

- **Hybrid Retrieval**: Incorporating sparse retrieval (BM25) to improve exact keyword matching for specific tickers and metrics.
- **Enhanced Table Modeling**: Moving beyond markdown/textual conversion toward JSON-based or hierarchical representations to preserve multi-dimensional headers.
- **Metadata-Gated Filtering**: Explicitly utilizing ticker and date metadata during the retrieval phase to eliminate cross-company semantic noise.

Ultimately, this work highlights that for the financial domain, retrieval is only as strong as the system's ability to maintain the integrity of the underlying structured data.

References

- Abdin, M. et al. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219.
- Chase, H. (2022). LangChain. GitHub. https://github.com/langchain-ai/langchain
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547.
- Lewis, P. et al. (2021). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401.
- Rajpurkar, P. et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. EMNLP 2016. arXiv:1606.05250.
- Richardson, L. (2007). Beautiful Soup Documentation. https://www.crummy.com/software/BeautifulSoup/
- Touvron, H. et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.
- Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020 System Demonstrations.
- Xiao, S. et al. (2023). C-Pack: Packaged Resources to Advance General Chinese Embedding. arXiv:2309.07597. (BAAI/bge model family)